



Sistemas Distribuidos

Profesor: Oscar Manuel Ruiz Hurtado

Evidencia S09

Que el aviso no tumbe la transferencia

Sierra García Mariana

Grupo: 02

Facultad de Ingeniería

Bitácora de Laboratorio:

1. Parte 0

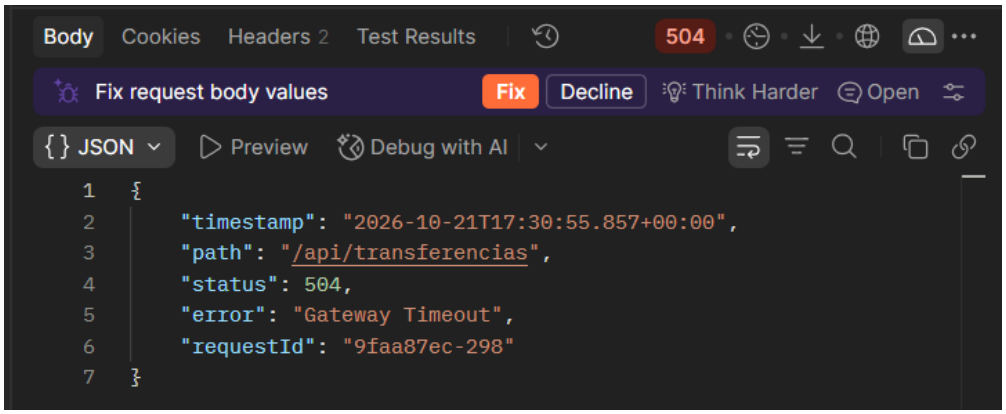


Figura 1: Error 504

¿Por qué pasó esto?
La transferencia se quedó en estado abonada y respondió con error porque al intentar notificar de forma síncrona por HTTP, el servicio destino estaba caído. La petición se quedó colgada esperando respuesta hasta que el API Gateway abortó por tiempo de espera a pesar de que el abono en base de datos ya había ocurrido.

2. Parte 1

En la arquitectura original el microservicio de transferencia dependía de una llamada HTTP síncrona hacia el servicio de notificacion. Este fuerte acoplamiento provocaba que ante la indisponibilidad del consumidor el sistema arrojara errores de tipo 500 o 503 por agotamiento del tiempo de espera. Esto interrumpía el flujo principal y devolvía un error al cliente a pesar de que las operaciones de cargo y abono ya se habían consolidado en la base de datos

Para aislar los fallos y garantizar la disponibilidad del core bancario y se refactorizó la lógica implementando un modelo de comunicación orientada a eventos mediante RabbitMQ, se definió el registro inmutable EventoTransferencia para encapsular el estado de la transacción y se sustituyó la dependencia del cliente REST por RabbitTemplate

la transferencia ahora se completa localmente y delega la responsabilidad de los avisos publicando un mensaje en el exchange:

```
EventoTransferencia evento = new EventoTransferencia(
    UUID.randomUUID(),
    "TRANSFERENCIA_REALIZADA",
```

```
    t.getId(),
    claveOrigen,
    claveDestino,
    monto,
    t.getSaldoOrigenTrasCargo(),
    destino.saldo(),
    Instant.now()
);

rabbitTemplate.convertAndSend("transferencias", "transferencia.realizada", evento);
```

Listing 1: Publicación del evento asíncrono utilizando RabbitTemplate

Para validar la tolerancia a fallos del nuevo diseño se detuvo intencionalmente el contenedor del servicio consumidor (docker compose stop notificacion) aislando la capa de mensajería. Al ejecutar la petición POST de prueba, el servicio de transferencias operó con normalidad y respondió exitosamente con un código HTTP 201 Created esto demuestra que la operación principal ya no se bloquea y el evento queda almacenado de forma segura en la cola de RabbitMQ en espera de que el consumidor recupere su disponibilidad

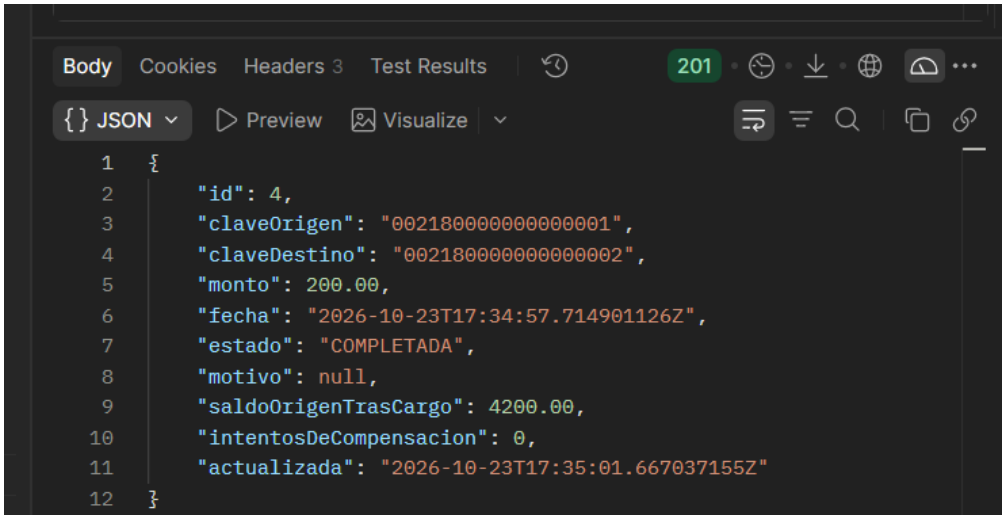


Figura 2: Validación de tolerancia a fallos

3. Parte 2

Para validar la correcta implementación del desacoplamiento asíncrono y la resiliencia del sistema se ejecutaron dos escenarios de prueba principales utilizando Postman y el panel de administración de RabbitMQ

Se simuló la caída del microservicio notificacion deteniendo su contenedor, durante este periodo de inactividad, se emitieron tres peticiones de transferencia exitosas al inspeccionar el broker de mensajería y se confirmó que RabbitMQ retuvo los eventos de forma segura en la cola notificacion.transferencias

al reiniciar el servicio el consumidor se reconectó y procesó los mensajes encolados inmediatamente, la



Figura 3: Captura de la interfaz de RabbitMQ mostrando 3 mensajes retenidos en estado Ready mientras el consumidor se encuentra inactivo.

consulta a la bandeja de notificaciones de la cuenta destino demostró la persistencia exitosa de los avisos de TRANSFERENCIA_RECIBIDA validando que no hubo pérdida de datos.

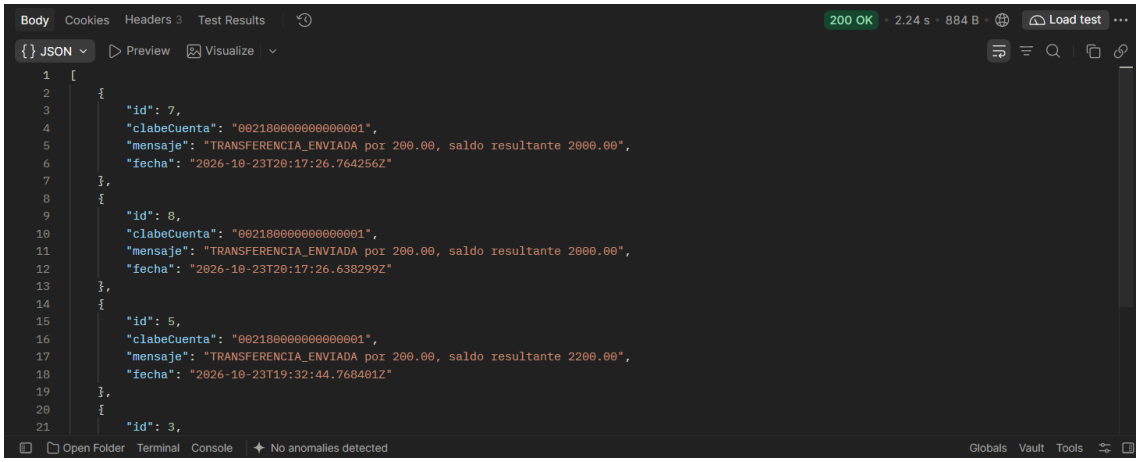


Figura 4: Respuesta HTTP 200 OK en Postman mostrando los avisos procesados asíncronamente al reiniciar el servicio de notificaciones

4. Parte 3

En los sistemas de mensajería asíncrona el patrón de entrega .ªl menos una vez"garantiza que los mensajes no se pierdan pero introduce el riesgo de procesamiento duplicado si un evento se reintenta o clona.

Para poder demostrar este comportamiento se detuvo intencionalmente el consumidor y se generó una transferencia. Utilizando el panel de administración de RabbitMQ, se extrajo el payload del mensaje retenido y se republicó manualmente en el exchange transferencias conservando el mismo eventoId. Al consultar la bandeja del cliente se observó que el sistema generó las notificaciones por duplicado para la misma transacción

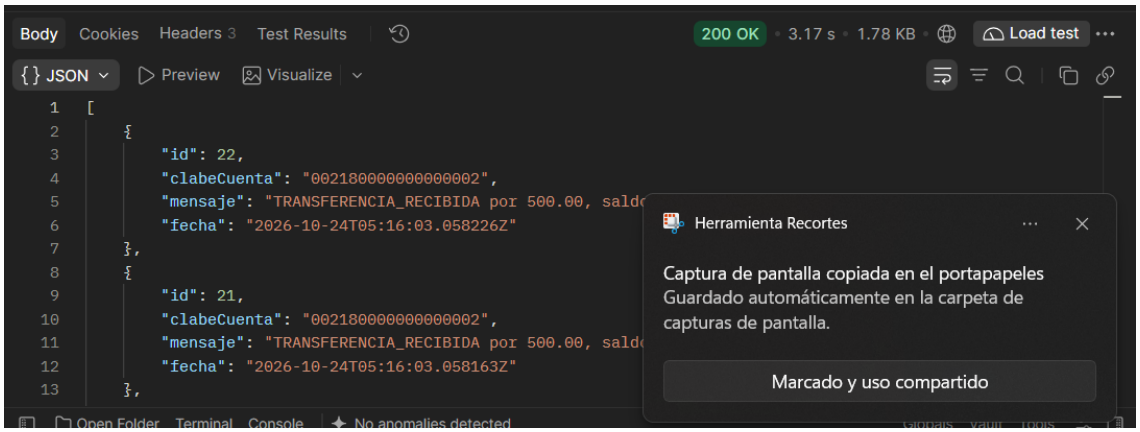


Figura 5: Respuesta de la API mostrando avisos duplicados tras la inyección manual de un mensaje clonado, evidenciando la falta de idempotencia inicial

Para lidiar con este problema se implementó un mecanismo de idempotencia en el consumidor de notificaciones. Se creó la entidad EventoProcesado para registrar el identificador único de cada mensaje procesado. Dentro de la misma transacción de base de datos el sistema ahora verifica si el identificador ya existe en caso afirmativo descarta la operación y confirma el mensaje a RabbitMQ para retirarlo de la cola

Al repetir el experimento de clonación e inyección manual con este nuevo mecanismo el sistema procesó la primera entrega y descartó exitosamente el duplicado resultando en un único juego de avisos en la base de datos

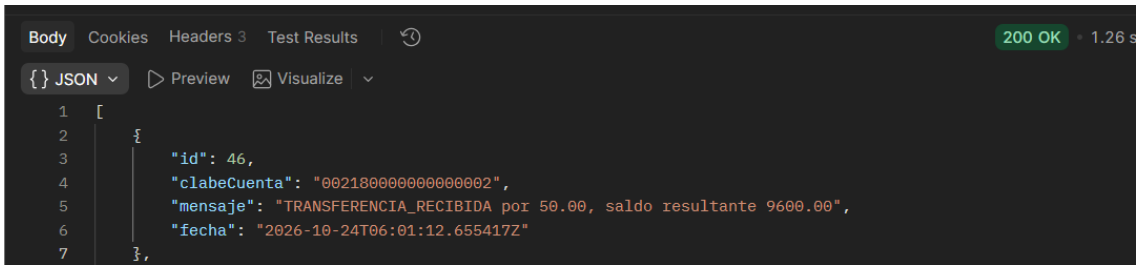


Figura 6: Bandeja de notificaciones mostrando un procesamiento único y correcto demostrando la efectividad de la restricción de idempotencia.

5. Parte 4

Un riesgo crítico en las arquitecturas orientadas a eventos es la recepción de mensajes malformados o "envenenados". Para simular esto se inyectó manualmente un payload JSON inválido ({"hola":) desde el panel de administración de RabbitMQ hacia el exchange principal

el sistema carecía de un mecanismo de desvío el error de deserialización en el consumidor provocó un rechazo estándar que reencoló el mensaje inmediatamente desencadenando un ciclo de reintentos infinito. Este comportamiento saturó los registros del contenedor y bloqueó el procesamiento de cualquier transferencia válida subsecuente

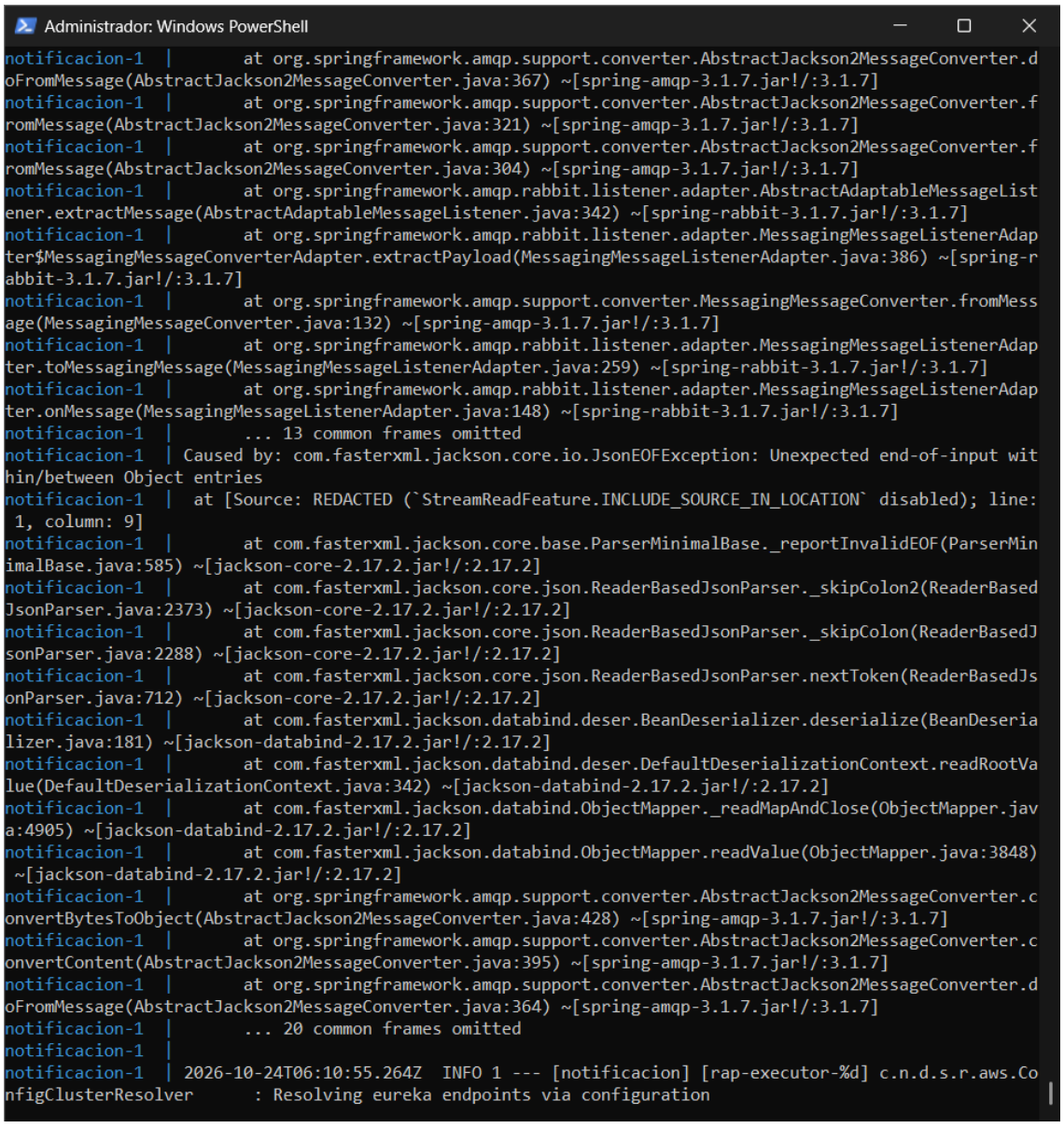


Figura 7: Ciclo infinito de excepciones en los registros del contenedor provocado por la lectura repetida de un mensaje malformado.

Para dotar al microservicio de resiliencia se implementó el patrón de Dead Letter Queue, se reconfiguró la

cola notificaciones.transferencias añadiendo los argumentos x-dead-letter-exchange y x-dead-letter-routing-key, se modificó el consumidor para interceptar el mensaje en formato crudo si el parseo falla, el sistema ahora emite un rechazo explícito sin reencolamiento

Al repetir la inyección del mensaje corrupto bajo esta nueva arquitectura el sistema aisló la falla exitosamente, el mensaje envenenado fue desviado de forma automática a la cola notificacion.transferencias.dlq, preservando la estabilidad del consumidor y garantizando el procesamiento ininterrumpido del tráfico regular

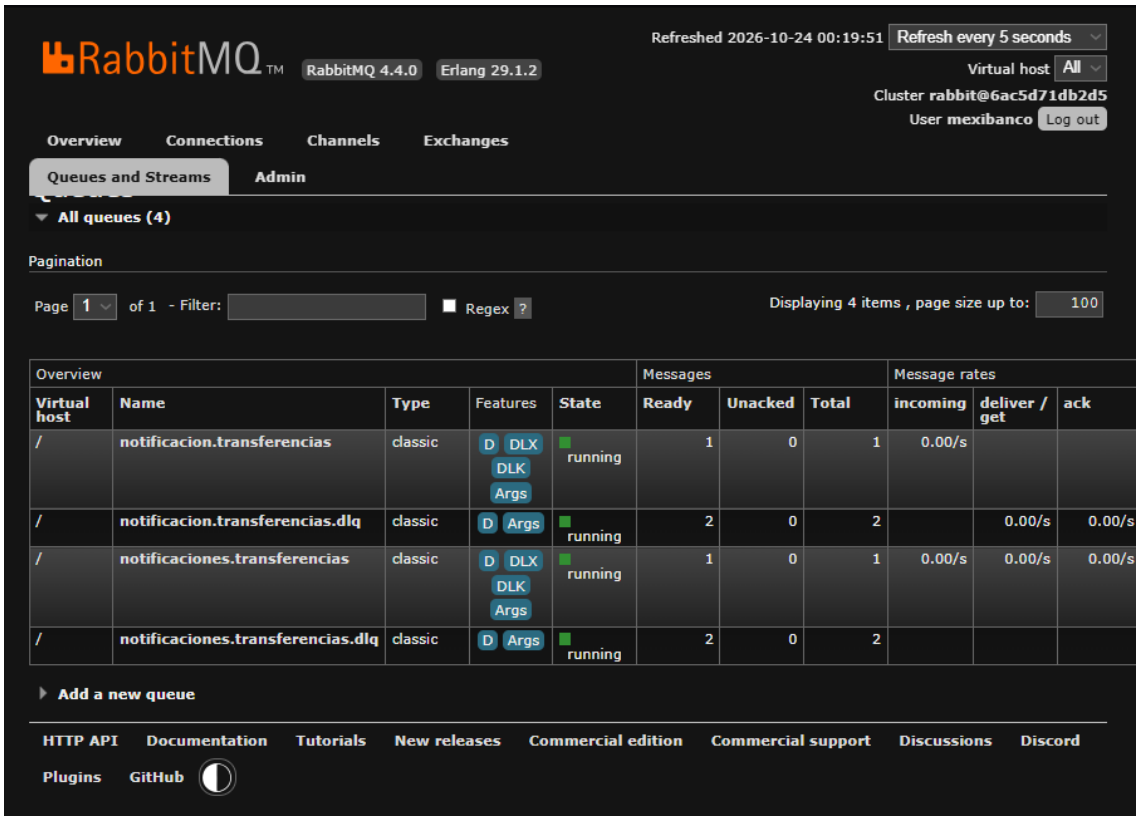


Figura 8: Panel de RabbitMQ confirmando la retención del mensaje envenenado en la DLQ evitando el colapso del consumidor principal

6. Preguntas de Cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?

En la versión actual de la arquitectura si RabbitMQ se encuentra caído en el momento exacto en que la saga intenta publicar el evento el mensaje no podrá encolarse y se perderá, el mecanismo de publisher confirm no soluciona este problema porque su única función es avisarle al microservicio productor si el broker recibió el mensaje o si falló, sin embargo no tiene memoria ni capacidad para guardar o reintentar automáticamente los mensajes cuando no hay conexión. Para arreglarlo definitivamente se requeriría el patrón Transactional Outbox

2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?
El acuse de recibo (ack) se ejecuta después de guardar en la base de datos para asegurar el patrón de entrega "al menos una vez", al hacerlo en este orden, garantizamos que si el servicio falla justo al escribir en la base de datos, el mensaje regresará a la cola y no se perderá. Por el contrario, si enviáramos el ack antes de guardar obtendríamos la garantía de que un mensaje jamás se procesará dos veces pero con el alto riesgo de perder la información para siempre si la base de datos falla inmediatamente después de haber confirmado su recepción a RabbitMQ

3. Elige y di qué sacrificaste:
Elección: Desacoplar el servicio de notificaciones usando mensajería asíncrona
Porque: Aumenta la disponibilidad y resiliencia del sistema principal si notificaciones cae, las transferencias no fallan y siguen operando
Sacrifico: La consistencia inmediata los usuarios ya no ven su saldo y avisos actualizados en tiempo real estricto sino bajo consistencia eventual